

Hack-Intern Challenge — UAV Track

Exercise Kestrel: signals analysis on a recorded drone link

Field	Value
Portal	https://13-200-126-136.sslip.io
Files	https://github.com/assist-ai-bot/UAV_CTF_v1
Lab signing key, for stage S4	b92d7e32a1e99249bd63803d6cea7ede67d4754506bd3f8be786314e9815efe9

The signing key is 64 hex characters on a single line. Copy it whole — if you drop characters off the end, every uplink signature will fail and stage S4 will look like one enormous forgery.

What this exercise is

You are given one recording of radio activity around a small survey drone, and you have to work out what happened during its flight. Everything you need is in that recording. Nothing else is required: no radio hardware, no internet lookups against live systems, and no transmitting of any kind.

The recording is a raw stream of radio samples, not a log file. That means the first part of the job is engineering: turn radio into bits, bits into messages. The second part is analysis: read those messages critically and decide what the evidence actually supports. Both halves are assessed, and the second one carries the most weight. We would rather see a careful analyst who reaches four well-evidenced conclusions than someone who guesses all 14 answers and cannot explain any of them.

Everything in the capture is synthetic. It was generated for this exercise. No real aircraft, operator or network is involved.

The situation

A lab survey multirotor — ArduPilot-style autopilot, MAVLink system id 1 — flew a test sortie. A passive receiver recorded the 915 MHz-band environment for the whole flight. Afterwards the lab noticed something wrong with the flight and suspects two things: that a command reached the aircraft from someone who was not the real ground station, and that its satellite navigation was interfered with.

Neither suspicion has been confirmed. That is your job. You are the analyst.

Your files

Everything is in the repository above. The recording is too large to commit, so it is attached to the **latest release** rather than sitting in the file tree.

File	Where	What it is
capture.cs8	Releases	The recording. 8-bit signed I/Q, interleaved, about 150 s long, 300 MB (300,000,000 bytes exactly). Check it against the SHA-256 below before you start.
capture.meta	repository	Recorder metadata supplied by the lab operator. Unverified. This recorder has a history of misreporting its own settings, and we have not confirmed any value in this file.
reference_clean_flight.json	sent 90 minutes in	Decoded telemetry from an earlier, clean flight of the same aircraft on the same profile, with no attack. Use it to establish what normal looks like.

capture.cs8 SHA-256 661f854868cd2ed88ef42f52481295a7205eaf20a850d2dd4e6cbb5a92397387

If the hash does not match, or the download looks short, tell the organisers before you start work rather than after. A truncated recording will waste your whole session.

What the lab can tell you

This is the complete set of facts we can vouch for. Anything not listed here, you determine yourself.

- The link uses **CC1101-class packet radios**. Packets are structured as preamble, sync word, length, payload, CRC-16, and the radio applies PN9 whitening to everything after the sync word. We do not have the CRC parameters — treat them as unknown.
- **Downlink** (aircraft to ground): GFSK, **50 kbaud**, one fixed channel on the low side of the recorder's centre frequency. Frames carry **MAVLink 2**, unsigned.
- **Uplink** (ground to aircraft): 2-FSK, **20 kbaud**, frequency-hopping across several channels on the high side of centre. Frames carry **signed MAVLink 2**. The signing key for this rig is printed at the top of this briefing.
- The recorder's centre frequency is nominally 915 MHz. Its sample rate is **not** reliably stated in `capture.meta`.
- Other transmitters share this band. Not everything in the recording belongs to the aircraft.
- Express every time you report as **seconds from the first sample of the capture**. If you work from vehicle timestamps, convert them.

Read this twice

Some frames and strings in this capture look like valid answers and are not. They are there deliberately. Anything that fails an integrity check — at any layer — is not evidence, and submitting something recovered from a frame that failed its own checksum will cost you the stage.

The six stages, and exactly what each one wants

Stages run in order, because each needs the output of the last. Stage S1 is a gate: if your demodulator is not exactly right, nothing after it will be either.

For every stage below you get the task list and the exact answer format. The examples in the last column show the **shape** of a correct answer — the values in them are made up.

How to write an answer

Answers are checked literally. Keep the `KST{...}` braces, use lower case, put no spaces anywhere, and give every time as whole seconds counted from the first sample of the recording.

S0 — Survey the band

Work out what you are actually holding before you decode anything.

What you must do

1. Measure the recording's **true sample rate**. Do not trust capture .meta; it is wrong, and every later measurement depends on this.
2. Find the drone's downlink and measure its centre, as an offset from the recorder's centre frequency, in **kHz**, **rounded to a whole number**. It is on the low side, so the number is negative.
3. Count how many separate channels the hopping uplink uses.
4. Account for the other transmitters you can see. They are not the drone, and some of them are there to waste your time.

Submit	What it is	Exact format	Example of the format
S0	Band inventory	KST{inv:<kHz>:<channels>:<baud>}	KST{inv:-123:4:50000}

<baud> is the downlink symbol rate, which this briefing has already told you. It is there so your answer shows all three facts together.

S1 — Get bytes off the downlink

The whole exercise rests on this stage, and it is graded all-or-nothing on purpose.

What you must do

1. Find every downlink burst in the recording.
2. Work out the link settings the lab could not give you, and demodulate each burst into bytes.
3. Keep only the frames whose **radio CRC passes**, and count them.
4. Join those frames' payloads together in time order and take the SHA-256 of the result. "Payload" means the bytes after the length byte and before the CRC, after you have undone the whitening.

Submit	What it is	Exact format	Example of the format
S1a	How many frames pass the radio CRC	KST{frames:<N>}	KST{frames:1234}
S1b	First 8 hex characters of that SHA-256	KST{sha:<8 hex>}	KST{sha:0a1b2c3d}

S2 — Read the telemetry

Three messages have been hidden inside ordinary-looking telemetry by someone with access to the aircraft. Each is a complete KST{ . . . } string once you reassemble it.

What you must do

1. Parse your frames as MAVLink 2, with message validation left switched on.
2. One message was too long for a single frame and was sent in pieces. Put it back together.
3. One message is hidden in the **names** of reported values rather than in the values.
4. One message is hidden in a numeric field of the heartbeat that should never hold a large number.

Submit	What it is	Exact format	Example of the format
S2a	The message in the status text	the KST{...} string	KST{abc123xyz}
S2b	The message in the named values	the KST{...} string	KST{abc123xyz}
S2c	The message in a heartbeat field	the KST{...} string	KST{abc123xyz}

S3 — Look beyond the bytes

Two more messages are not carried by any field of any message. Decoded bytes alone will not give you either of them.

What you must do

1. One message is in **when** transmissions happen rather than in what they say. You will need the time each burst started, measured from the recording.
2. One message comes from a separate, very weak transmitter. You will not see it in a casual look at the spectrum, and turning up the gain will not help.

Submit	What it is	Exact format	Example of the format
S3a	The message carried by timing	the KST{...} string	KST{abc123xyz}
S3b	The message from the weak emission	the KST{...} string	KST{abc123xyz}

S4 — Audit the uplink

Somebody sent this aircraft a command. Work out who, and what the aircraft did about it.

What you must do

1. Decode the hopping uplink. Every 40 ms it moves to a different channel.
2. Number every uplink MAVLink frame **0, 1, 2, ... in time order, including the ground station's heartbeats**. S4a and S4b both refer to this numbering, so get it right first.
3. Check each frame's signature against the lab key printed at the top of this briefing. Exactly one frame is signed with a different key. Note which parameter it set.
4. Find the frames that are genuine but **not fresh** — byte-for-byte repeats of commands sent earlier.
5. Work out the hop pattern well enough to say which channels the **next 20 dwells** would have used, had the recording continued. Channel 0 is the lowest-frequency uplink channel.

Submit	What it is	Exact format	Example of the format
S4a	The forged frame and the parameter it set	KST{forged:<index>:<PARAM>}	KST{forged:12:SOME_PARAM}
S4b	Indices of the replayed frames, ascending	KST{replays:<i>,<j>,<k>}	KST{replays:3,17,42}
S4c	The next 20 channel indices, as 20 digits	KST{hops:<20 digits>}	KST{hops:01234567012345670123}

S5 — Was the navigation attacked?

Decide whether satellite navigation was interfered with, and if so, characterise it. The clean reference flight released at T+90 is the same aircraft on the same profile with no attack — use it to establish what normal looks like.

What you must do

1. Compare the reported GPS position against things the attacker does not control. One source of truth is the aircraft's own blended navigation solution, which it reports separately.
2. Decide **when** the attack began, in whole seconds from the first sample. Anything within 2 seconds of the true answer is accepted.
3. Decide **what kind** of attack it was: dragoff (the false position is pulled away slowly), jump (it moves in one step) or meaconing (genuine signals rebroadcast with a delay).
4. Decide **when the aircraft's navigation filter accepted the false position as true**. Within 1 second is accepted.

Submit	What it is	Exact format	Example of the format
S5a	When the attack began (whole seconds)	KST{onset:<s>}	KST{onset:42}
S5b	What kind of attack it was	KST{class:<name>}	KST{class:jump}
S5c	When the filter accepted it (whole seconds)	KST{reset:<s>}	KST{reset:99}

A warning about stage S5

S5 is built so that no single measurement looks obviously wrong at the moment the attack starts, and so that the most obvious indicator goes quiet later in the flight, exactly when the attack is succeeding. If you analyse only the last part of the recording, the aircraft will look perfectly healthy.

Suggested toolkit

Nothing here is mandatory and nothing here is complete — use whatever you are fastest in. This list exists so that no one loses an afternoon to tooling. It is deliberately generic: naming the right algorithm for each stage would be most of the answer, so you will not find that here.

Core environment

Tool	Why
Python 3.10+	Assumed by everything below. A virtual environment keeps this reproducible for graders.
NumPy	Bulk sample arithmetic. Use <code>np.memmap</code> so you are not holding the whole recording in RAM.
SciPy	<code>scipy.signal</code> covers filter design, filtering, resampling, spectrograms and peak finding — most of the DSP you need.
Matplotlib	Look at your data. Most mistakes in this exercise are visible in a plot long before you can prove them in code.
Jupyter	Get one burst right interactively before you run three thousand of them in a batch.

Protocol

Tool	Why
pymavlink	The MAVLink 2 reference implementation. Set <code>MAVLINK20=1</code> and use the <code>ardupilotmega</code> dialect. Decode with message-integrity validation enabled; tolerant or “robust” parsing modes will quietly hand you frames you should have rejected.
mavlogdump.py, mavextra	Ship with pymavlink. Useful for sanity-checking and quick plots once you have bytes — but note that not every answer in this exercise is reachable from decoded messages alone.

Looking at the radio by hand

Tool	Why
inspectrum	Opens raw interleaved 8-bit files directly. Zoom into a single burst, measure offsets and symbol rates with the cursors, and eyeball the framing. Usually the fastest route to understanding a burst before you automate it.
Universal Radio Hacker (URH)	Interactive demodulation and bit-level inspection of unknown FSK/OOK framing. Good for a first pass at an unfamiliar packet structure.
GNU Radio, gnuradio-companion	Flowgraph prototyping if you prefer building a receive chain visually to writing it in NumPy.
gqrx, SDR++	Optional. Mostly oriented to live radios; of limited use on a file, but fine for a quick look.

Bits, framing and integrity

Tool	Why
hashlib, hmac	Standard library. You need these for the hash answer in S1 and for the proof values in your reproducibility package.
crcmod, or your own	A CRC-16 is about twenty lines. Writing it yourself makes it easy to sweep a small space of configurations, which you will need to do.
np.packbits / np.unpackbits, bitstring	Bit-to-byte conversion without off-by-one errors. Worth using rather than hand-rolling, because an off-by-one here looks exactly like a broken demodulator.

Analysis and bookkeeping

Tool	Why
pandas	Optional, but convenient for time-series telemetry and for comparing against the clean reference flight.
sympy, galois	If you find yourself needing to model a deterministic sequence generator.
SigMF	A tidy way to record what you concluded about the recording's true parameters, and to keep that conclusion attached to the file.
git	Graders re-run your package. Commit the exact version that produced the answers you submitted.

Hardware

None. This is an offline analysis exercise on a recorded file. Nothing in it requires a radio, and nothing in it should lead you to transmit. If you think a stage needs you to transmit, re-read the stage.

Getting started

A sanity check that reads the file, removes the DC offset and plots a spectrogram of a slice. If this runs and you see structure, your tooling is fine and the exercise is open to you.

```
import numpy as np, matplotlib.pyplot as plt
from scipy import signal

# cs8 = interleaved signed 8-bit I,Q. memmap: do not pull 300 MB into RAM.
x = np.memmap('capture.cs8', dtype=np.int8).reshape(-1, 2)
print(x.shape[0], "IQ samples")

# DC offset is significant here. Estimate it once, subtract it everywhere.
dc = x[:2_000_000].astype(np.float32).mean(0)
print("DC offset (I, Q):", dc)

# Work on slices, not the whole file: complex128 over 150 M samples is gigabytes.
seg = x[:5_000_000].astype(np.float32)
iq = (seg[:, 0] - dc[0]) + 1j * (seg[:, 1] - dc[1])

# FS_TRUE is yours to establish. capture.meta states a rate; it is not confirmed.
f, t, Sxx = signal.spectrogram(iq, fs=FS_TRUE, return_onesided=False,
                                nperseg=1024, noverlap=0)
plt.pcolormesh(t, np.fft.fftshift(f) / 1e3,
               10 * np.log10(np.fft.fftshift(Sxx, axes=0) + 1e-12), shading="auto")
plt.ylabel("kHz from centre"); plt.xlabel("s"); plt.colorbar(label="dB")
plt.show()
```

Practical cautions

- The recording is 300 MB (300,000,000 bytes exactly) of samples. A single fine-resolution transform over the whole file will not fit comfortably in memory — process it in chunks and accumulate.
- Subtract the DC offset before you measure anything. It is large enough to bias frequency estimates.
- A weak signal needs **integration**, not gain. Amplifying noise and signal together changes nothing.
- Check integrity at every layer, and keep the layers distinct: a frame can pass the radio's check and still fail the protocol's, and the two failures mean different things.

- Do not throw away burst timestamps once you have bytes. Not everything in this capture is carried by the bytes.
- Each challenge allows **5 attempts**. Verify before you submit; the decoys are there to be fallen for.

All 14 answers at a glance

Every answer, in one place, for when you are filling the portal in. Submit each through its own CTFd challenge. Substitute your values for the angle-bracketed parts and change nothing else.

Submit	What it is	Exact format
S0	Band inventory: carrier offset in kHz, uplink channel count, downlink baud	KST{inv:<kHz>:<channels>:<baud>}
S1a	Downlink frames passing the radio CRC	KST{frames:<N>}
S1b	First 8 hex of the SHA-256 of those payloads, in time order	KST{sha:<8 hex>}
S2a	Message hidden in the status text	the KST{...} string
S2b	Message hidden in the named values	the KST{...} string
S2c	Message hidden in a heartbeat field	the KST{...} string
S3a	Message carried by transmission timing	the KST{...} string
S3b	Message from the weak narrowband emission	the KST{...} string
S4a	Forged frame index and the parameter it set	KST{forged:<index>:<PARAM>}
S4b	Replayed frame indices, ascending	KST{replays:<i>,<j>,<k>}
S4c	Channels of the next 20 dwells after the recording ends	KST{hops:<20 digits>}
S5a	When the GNSS attack began, whole seconds (± 2 s)	KST{onset:<s>}
S5b	Attack class: dragoff, jump or meaconing	KST{class:<name>}
S5c	When the navigation filter accepted it, whole seconds (± 1 s)	KST{reset:<s>}

Each challenge allows a limited number of attempts, so check an answer before you spend one. Some strings in the recording look like valid answers and are not.

How you are assessed

The flags are the easy half, and on their own they settle nothing — an answer string proves you arrived somewhere, not that you know how. Submit the flags on the portal as you get them, and send the written work afterwards by **replying to the email that invited you**.

	What to send	When
Flags	Each answer through its own challenge on the portal, under UAV Track . Nothing to email.	as you solve them
Written work	The four items below, in one reply to your invitation email.	within 24 hours of the portal closing

#	Deliverable
1	Decision memo per stage. For each stage: your claim, the evidence for it and how to reproduce that evidence, the alternatives you ruled out and how you ruled them out, your confidence from 0–100%, and what would change your mind.

#	Deliverable
2	Reproducibility package. Your code plus a README. One command, run against <code>capture.cs8</code> , must print the answers you submitted. We will run it, so pin your dependencies and say which Python you used. Code that only works on your machine counts for very little.
3	Verification log. Where you used AI assistance, what it got wrong, and how you caught it. This is assessed as a skill, not confessed as a lapse.
4	Capstone, one page. Tie the uplink event and the navigation event into a single timeline. State the limitations of your analysis, and give two ranked mitigations.

A candidate who submits six flags and explains them well will place above one who submits all fourteen and cannot.

Rules

- AI coding assistants and any software tools are **allowed and expected**. Document your use of them in deliverable 3.
- Other people are not allowed. Do not share the files, your answers or your approach with anyone.
- Do not publish this exercise, the capture, or your solution afterwards.
- Offline analysis only. Do not transmit, and do not probe any live system.
- If a file fails its SHA-256 check or a download looks truncated, contact the organisers before you start work rather than after.

Everything in this capture is synthetic and was generated for this exercise. The scenario is lab-scaled — the navigation event unfolds considerably faster than a real one would, so that it fits inside 150 seconds. Judge the evidence on its own terms.